

-  Secrets
-  ABAP
-  Ansible
-  Apex
-  AzureResourceManager
-  C
-  C#
-  C++
-  CloudFormation
-  COBOL
-  CSS
-  **Dart**
-  Docker
-  Flex
-  Go
-  HTML
-  Java
-  JavaScript
-  JCL
-  Kotlin
-  Kubernetes
-  Objective C
-  PHP
-  PL/I
-  PL/SQL
-  Python
-  RPG
-  Ruby
-  Scala
-  Swift
-  Terraform
-  Text
-  TypeScript
-  T-SQL
-  VB.NET
-  VB6
-  XML



Dart static code analysis

Unique rules to write Clean DART Code

All rules **115**

 Bug **15**

 Code Smell **100**

Tags ▾

Impact ▾

Clean code attribute ▾

Search by name... 🔍

Local identifiers should not start with underscore

 Code Smell

If-null operator should be preferred

 Code Smell

Null-aware operators should be preferred

 Code Smell

Generic function type aliases should be preferred

 Code Smell

Type parameters should not shadow other type parameters

 Code Smell

"static final" declarations should be "const" instead

 Code Smell

Interpolation should be used instead of String concatenation

 Code Smell

Collection literals should be preferred

 Code Smell

Conditional assignment should be preferred

 Code Smell

"this" should only be used when required

 Code Smell

Exceptions should not be ignored

 Code Smell

Code annotated as deprecated should not be used

Local identifiers should not start with underscore

Analyze your code

Consistency - Identifiable

Maintainability ▾

 Code Smell

 Minor

 convention

In Dart, local variables don't have a concept of "private visibility" and should not start with an underscore.

Why is this an issue?

How can I fix it?

More Info

The underscore prefix is used in Dart to indicate members and top-level declarations that are private.

The compiler enforces access restrictions so that private elements can only be accessed from within the library in which they are declared.

Because local variables and parameters are only accessible within the scope in which they are declared, and cannot be exposed to other libraries, the private accessibility scope doesn't apply to them. Using an underscore prefix for local variables suggests otherwise and can be misleading.

This rule applies to local variables and parameters of:

- top-level functions
- nested functions and lambdas
- class methods
- class constructors
- extension and extension type methods

Exceptions

Unused parameters are typically named `__`, `___`, etc. to indicate that they are intentionally unused. This is a common practice and is not considered a violation of this rule.

Typical examples of this practice arise with lambdas:

```
List<int> repeat(int value, int times) =>
  List<int>.generate(times, (_) => value); // OK
```

as well as in pattern matches with **wildcards**:

```
String rating(double probability) => switch (probability) {
  < 1.05 => 'A',
  < 20.0 => 'B',
  < 50.0 => 'C',
  _ => 'D', // OK
};
```

as well as catch clauses:

```
try {
  // ...
} on Exception catch (_, s) { // OK
  print(s);
}
```

Available In:

sonarcloud 

